

Guía del Equipo — App de Reportes Ambientales

Stack: Laravel 12 · React Native Expo · Docker · MySQL · GitHub
Proyecto universitario — Universidad San Sebastián

Equipo y roles

Nombre	Rol	Responsabilidad
Bastian	Líder + Backend	API Laravel, Docker, revisión de PRs, administración del repo
Mathias	Backend	Endpoints de autenticación y reportes
Sebastian	Backend	Migraciones, modelos, endpoint heatmap
Alonso	Frontend	Pantallas de autenticación (Login, Registro)
Catalina	Frontend	Pantalla de mapa y heatmap
Oskar	Frontend	Formulario de reporte (GPS, cámara, envío)

Estructura del repositorio

```
proyecto-reportes/  
├── backend/                → API Laravel 12  
│   ├── docker-compose.yml  
│   ├── Dockerfile  
│   ├── routes/api.php  
│   └── ...  
├── frontend/              → App React Native Expo  
│   ├── src/  
│   │   ├── screens/  
│   │   ├── components/  
│   │   └── services/api.js  
│   └── ...  
├── docs/  
│   ├── API.md             → contrato de endpoints  
│   └── database.png        → diagrama de BD  
├── README.md  
└── CLAUDE.md
```

Cómo clonar el proyecto (todos)

```
git clone https://github.com/Baztzy/proyecto-reportes.git  
cd proyecto-reportes  
git checkout develop
```

Setup Backend (Bastian, Mathias, Sebastian)

Requisitos

- Docker Desktop instalado y corriendo

Primera vez (setup inicial)

```
# 1. Copiar el archivo de entorno
cp backend/.env.example backend/.env

# 2. Abrir backend/.env con cualquier editor y completar:
#   DB_PASSWORD=la_contraseña_que_elijas
#   DB_ROOT_PASSWORD=otra_contraseña

# 3. Levantar los contenedores (puede tardar unos minutos la primera vez)
cd backend
docker compose up -d --build

# 4. Generar la clave de la app (solo una vez)
docker compose exec app php artisan key:generate

# 5. Publicar configuración de Sanctum (solo una vez)
docker compose exec app php artisan vendor:publish --provider="Laravel\Sanctum\SanctumService"

# 6. Crear las tablas en la base de datos
docker compose exec app php artisan migrate
```

Uso diario

```
# Encender los contenedores
cd backend && docker compose up -d

# Apagar los contenedores
cd backend && docker compose down
```

URLs disponibles

	Qué	URL
API		http://localhost:8000
phpMyAdmin (gestor visual de BD)		http://localhost:8080

En phpMyAdmin entra con el usuario y contraseña que pusiste en el .env

Comandos útiles de Laravel

```
# Crear una migración nueva
docker compose exec app php artisan make:migration create_reports_table

# Crear un modelo + migración + controlador en un solo paso
docker compose exec app php artisan make:model Report -mc

# Correr migraciones
docker compose exec app php artisan migrate

# Revertir y volver a correr migraciones (útil en desarrollo)
docker compose exec app php artisan migrate:fresh

# Correr los tests
docker compose exec app php artisan test
```

Setup Frontend (Alonso, Catalina, Oskar)

Requisitos

- Node.js instalado
- App **Expo Go** instalada en el celular (iOS o Android)

Primera vez (setup inicial)

```
cd frontend
npm install
```

Uso diario

```
cd frontend
npx expo start
```

Aparece un QR en la terminal. Escanéalo con la app **Expo Go** desde tu celular (deben estar en la misma red WiFi).

Estructura del código frontend

```
frontend/src/
├── screens/      → una pantalla por archivo (LoginScreen.js, MapScreen.js, etc.)
├── components/  → componentes reutilizables (botones, inputs, etc.)
├── services/
│   └── api.js    → todas las llamadas al backend van acá
```

Cómo llamar al backend desde el frontend

El archivo `src/services/api.js` ya tiene configurado axios. Úsalo así:

```
import api, { setAuthToken } from '../services/api';

// Login
const response = await api.post('/login', { email, password });
const token = response.data.token;
setAuthToken(token); // guarda el token para peticiones siguientes

// Obtener reportes
const reportes = await api.get('/reports');

// Crear un reporte
await api.post('/reports', {
  description: 'Basura en la esquina',
  latitude: -33.45,
  longitude: -70.65,
  photo: archivo,
});
```

Flujo de trabajo Git (todos)

Regla de oro

Nadie programa sin tener su Issue abierto y su rama feat/ creada.

Ciclo por tarea

```
# 1. Siempre partir desde develop actualizado
git checkout develop
git pull origin develop

# 2. Crear tu rama con el nombre de la tarea
git checkout -b feat/login-api      # ejemplo backend
git checkout -b feat/pantalla-login # ejemplo frontend

# 3. Programar y hacer commits frecuentes
git add .
git commit -m "feat: formulario de login con validación"

# 4. Subir tu rama
git push origin feat/nombre-tarea

# 5. Ir a GitHub → abrir Pull Request hacia develop
# 6. Esperar que al menos 1 compañero lo revise
# 7. Merge y eliminar la rama
```

Estructura de ramas

```
main ← solo código estable (via PR desde develop)
├── develop ← integración del equipo (via PR desde feat/*)
│   ├── feat/login-api
│   ├── feat/pantalla-login
│   └── feat/mapa-reportes
```

Formato de commits (obligatorio)

Prefijo	Cuándo usarlo
feat:	Nueva funcionalidad
fix:	Corrección de un bug
docs:	Cambios en documentación
chore:	Configuración, dependencias
refactor:	Mejorar código sin cambiar su comportamiento

Ejemplos:

```
feat: endpoint POST /api/login con Sanctum
fix: corregir validación de coordenadas GPS
feat: pantalla de mapa con marcadores
docs: actualizar contrato en API.md
```

API — Endpoints disponibles

Todos los endpoints protegidos requieren el header:

```
Authorization: Bearer {token}
```

Método	Endpoint	Descripción	Auth
POST	/api/register	Registrar usuario	No
POST	/api/login	Login, devuelve token	No
POST	/api/logout	Cerrar sesión	Sí
GET	/api/reports	Listar todos los reportes	Sí
POST	/api/reports	Crear reporte con foto	Sí
GET	/api/reports/{id}	Ver detalle de un reporte	Sí
GET	/api/reports/heatmap	Puntos para mapa de calor	Sí
GET	/api/user/reports	Reportes del usuario autenticado	Sí

Las fotos se envían como multipart/form-data. El backend devuelve una URL pública en `photo_url`.

Base de datos

Tabla	Campos principales
users	id, name, email, password, created_at
reports	id, user_id, description, latitude, longitude, photo_url, status, created_at
categories	id, name

Status de un reporte: pendiente → en revisión → resuelto

Roadmap semanal

Semana	Hito	Backend	Frontend
1	Setup	Docker + Laravel corriendo, /api/ping	Expo corriendo en celular
2	Contrato	API.md completo + migraciones	Pantallas wireframe sin lógica
3-4	Auth	Register / Login / Logout con Sanctum	Pantallas Login y Registro conectadas
5	Reportes	CRUD reportes + subida de fotos	Formulario con GPS y cámara
6	Mapa	Endpoint heatmap	Mapa con marcadores y heatmap
7	QA	Corrección de bugs	APK generado, pruebas en dispositivo

Preguntas frecuentes

¿Dónde escribo mi código?

- Backend → dentro de `backend/app/Http/Controllers/` (controladores) y `backend/database/migrations/` (tablas)
- Frontend → dentro de `frontend/src/screens/` (pantallas) y `frontend/src/services/` (llamadas a la API)

¿Por qué no subo `vendor/` ni `node_modules/`? Porque pesan demasiado y no hace falta. Al clonar el repo solo corres `composer install` o `npm install` y se descargan solos.

¿El `.env` va al repo? No. Cada developer tiene su propio `.env` con sus contraseñas. Solo el `.env.example` (sin contraseñas) va al repo.

¿Puedo subir directo a main o develop? No. Siempre trabajas en una rama feat/ y abres un Pull Request. Nadie sube directo.

¿Qué pasa si hay conflictos en Git? Avísale a Bastian antes de intentar resolverlos solo.